

Evaluation première semaine projet

1. Introduction

- **Besoins** : Gérer les espaces office (bureaux, salles, cabines) et permettre aux collaborateurs de les réserver.
- **Cible** : Entreprises nécessitant une solution de gestion de bureaux.
- **Contraintes** : Disponibilité dégradée préférable à une indisponibilité totale. Authentification JWT obligatoire. Annulation 24h minimum avant le début. Détection des chevauchements de créneaux.

2. Besoins fonctionnels « métier »

2.1. Utilisateurs du projet

Rôle	Processus impactés
Collaborateur (USER)	Consultation, réservation, annulation, check-in
Administrateur (ADMIN)	Gestion des espaces, supervision des réservations, audit
Système	Vérification JWT, notifications automatiques, logs d'audit

L'Administrateur hérite de tous les droits du Collaborateur.

2.2. Informations relatives aux contenus

- **Données utilisateurs** : email, mot de passe hashé, rôle, nom, prenom et civilite — soumises au RGPD
- **Données espaces** : nom, type, capacité, quota max, QR code
- **Données réservations** : créneaux, statut, participants, historique
- **Logs d'audit** : actions sensibles horodatées (MongoDB), non exposées publiquement

Pas de contenu médiatique ni de DRM. Une suppression de compte devra être prévue (RGPD).

2.3. Inventaire des besoins fonctionnels

Fonctionnalité	Collab.	Admin	Description
S'inscrire / Se connecter / Se déconnecter	✓	✓	Authentification JWT

Fonctionnalité	Collab.	Admin	Description
Consulter et filtrer les espaces	✓	✓	Par type, capacité, disponibilité
Créer une réservation (publique/privée)	✓	✓	Avec vérification de disponibilité et quota
Inviter des collaborateurs	✓	✓	Invitation par email, statut PENDING/ACCEPTED/DECLINED
Consulter ses réservations	✓	✓	Historique et à venir
Annuler sa réservation	✓	✓	Délai 24h, notification de tous les participants
Check-in QR code	✓	✓	Scanner le QR code de la salle pour confirmer la présence
Créer / Modifier / Supprimer un espace	—	✓	Suppression avec annulation + notification des réservations futures
Consulter toutes les réservations	—	✓	Vue globale de l'occupation
Tableau de bord & logs d'audit	—	✓	Statistiques d'occupation, traçabilité

3. UML

3.1. Diagramme de cas d'utilisation global

Scan QR code dans la salle
pour valider la présence physique

Vérifie qu'aucune réservation
existante ne chevauche les
créneaux demandés (startAt/endAt)

Réservation publique : visible par tous
Réservation privée : invitation uniquement

Témpo - Gestion d'espaces flex-office

Authentification

Se connecter

Se déconnecter

S'inscrire

Gestion des espaces

Consulter les espaces
disponibles

«extend»

Filtrer par type
(bureau / salle / cabine)

Vérifier la disponibilité
d'un espace

Créer un espace

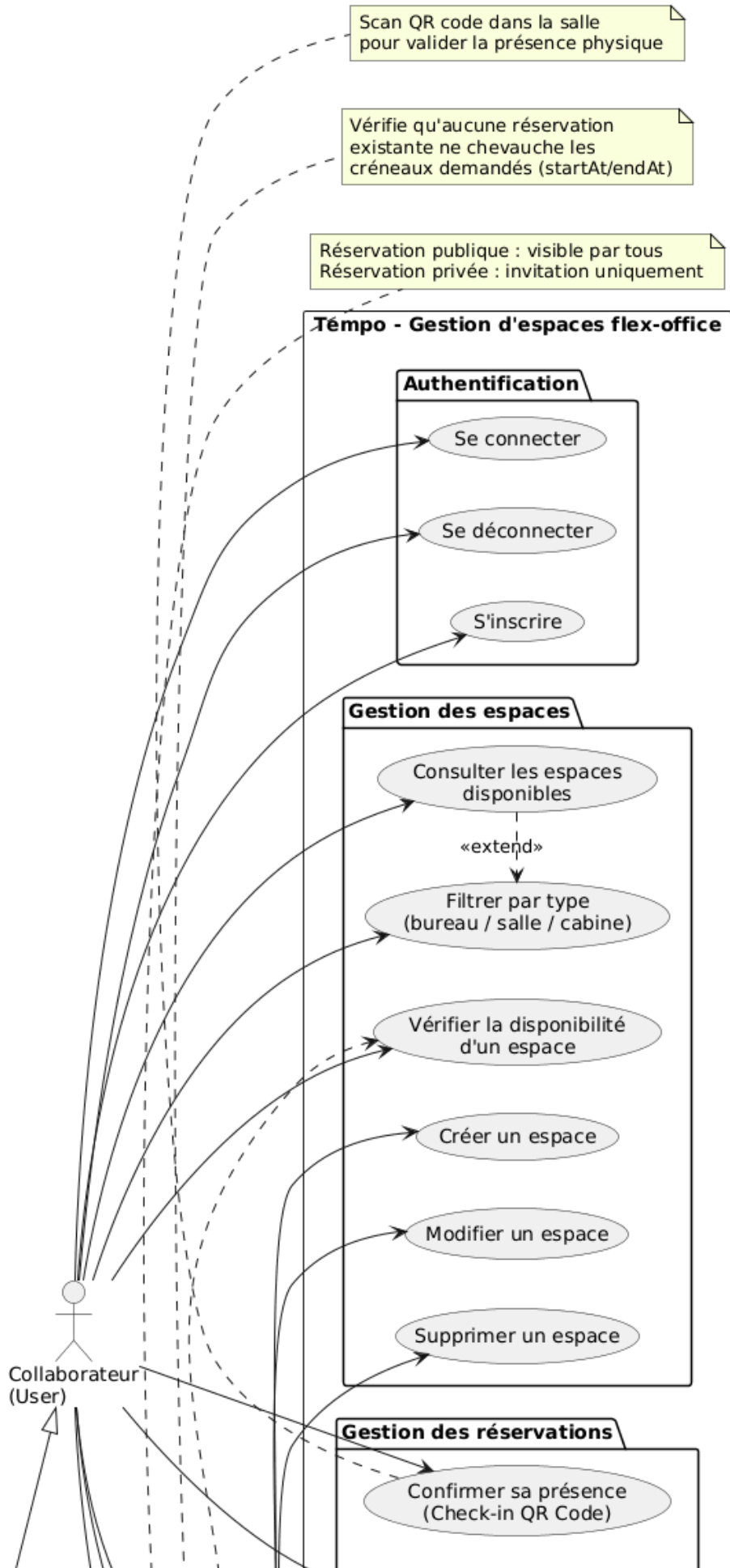
Modifier un espace

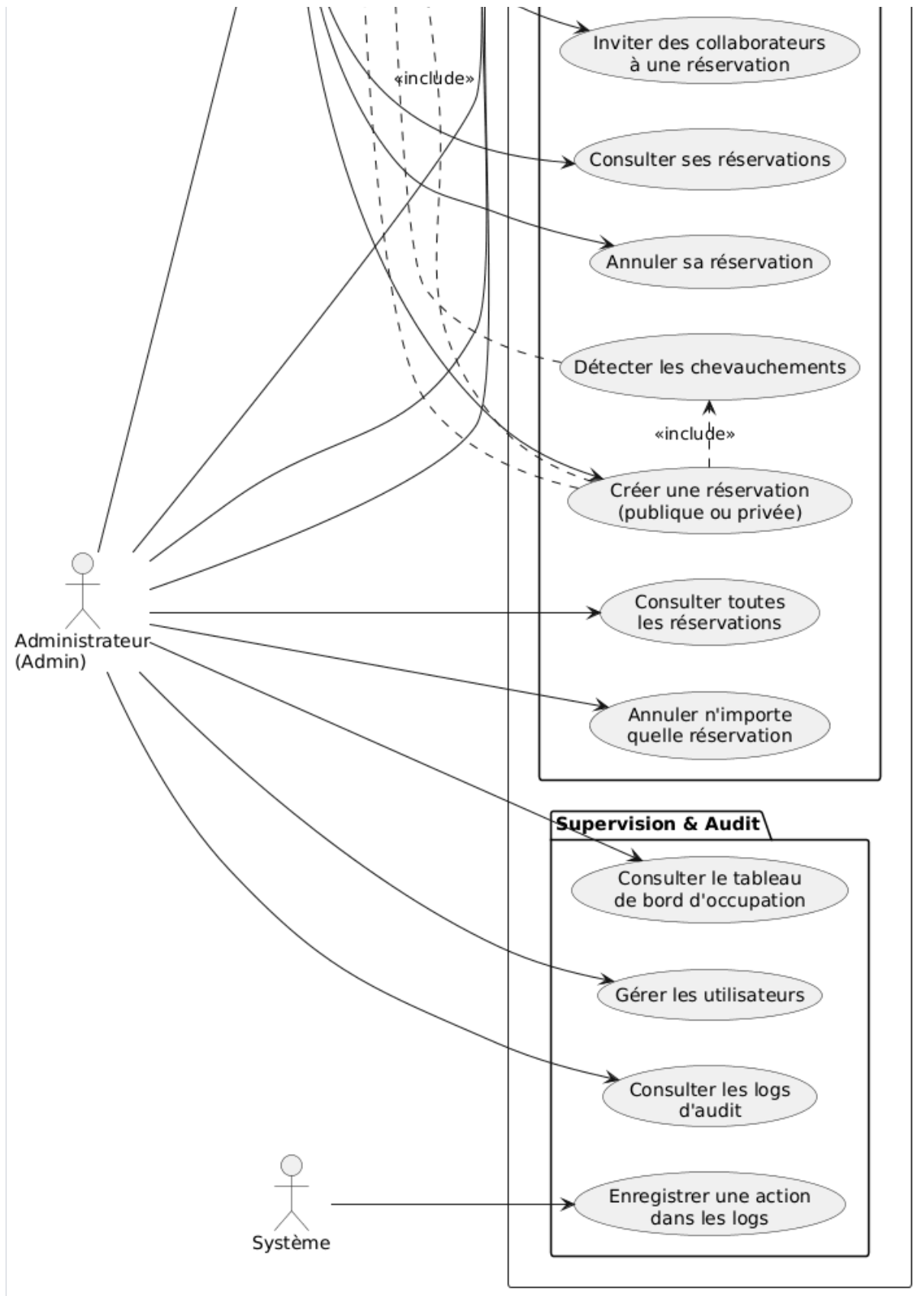
Supprimer un espace

Gestion des réservations

Confirmer sa présence
(Check-in QR Code)

Collaborateur
(User)



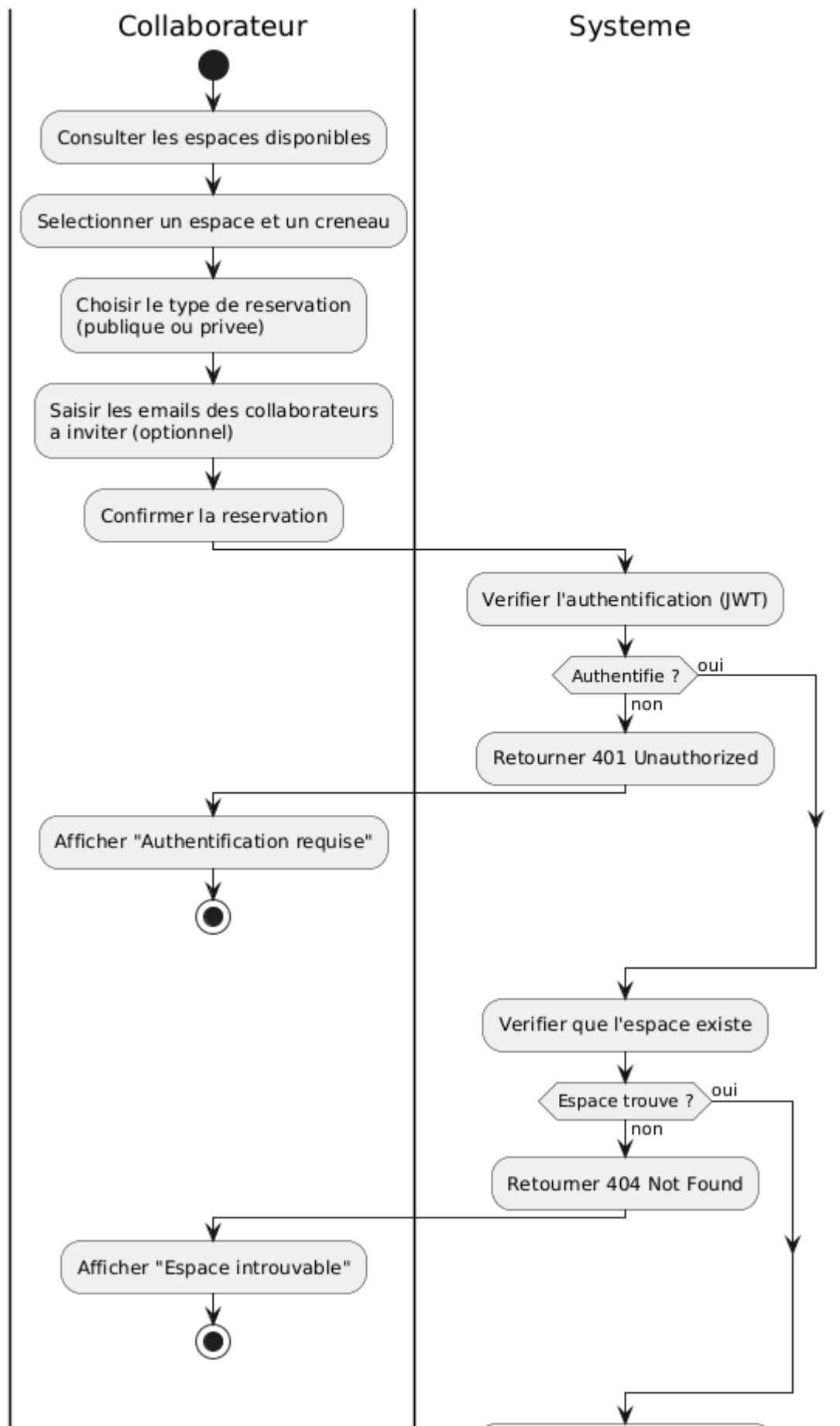


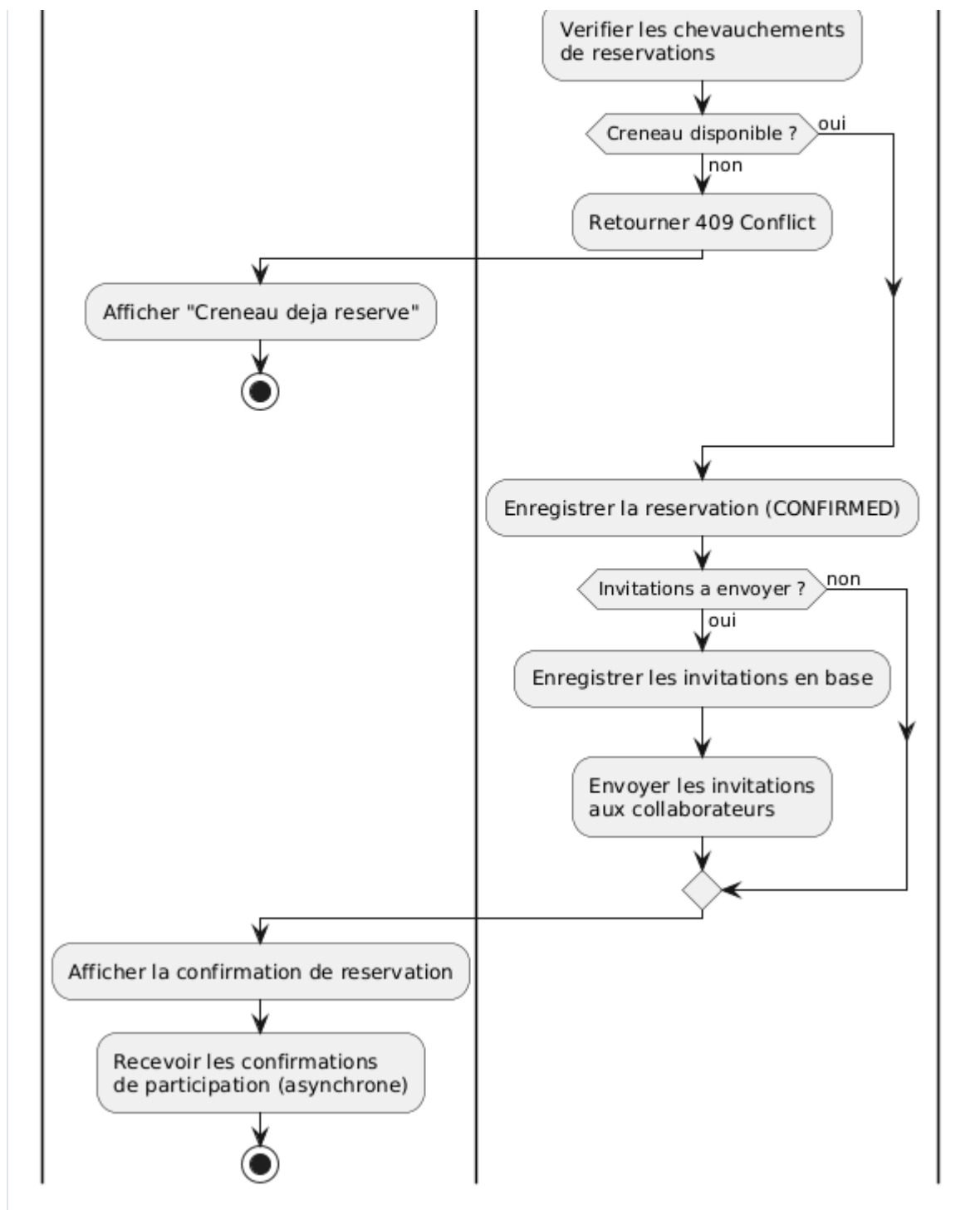
3.2. Fonctionnalités détaillées

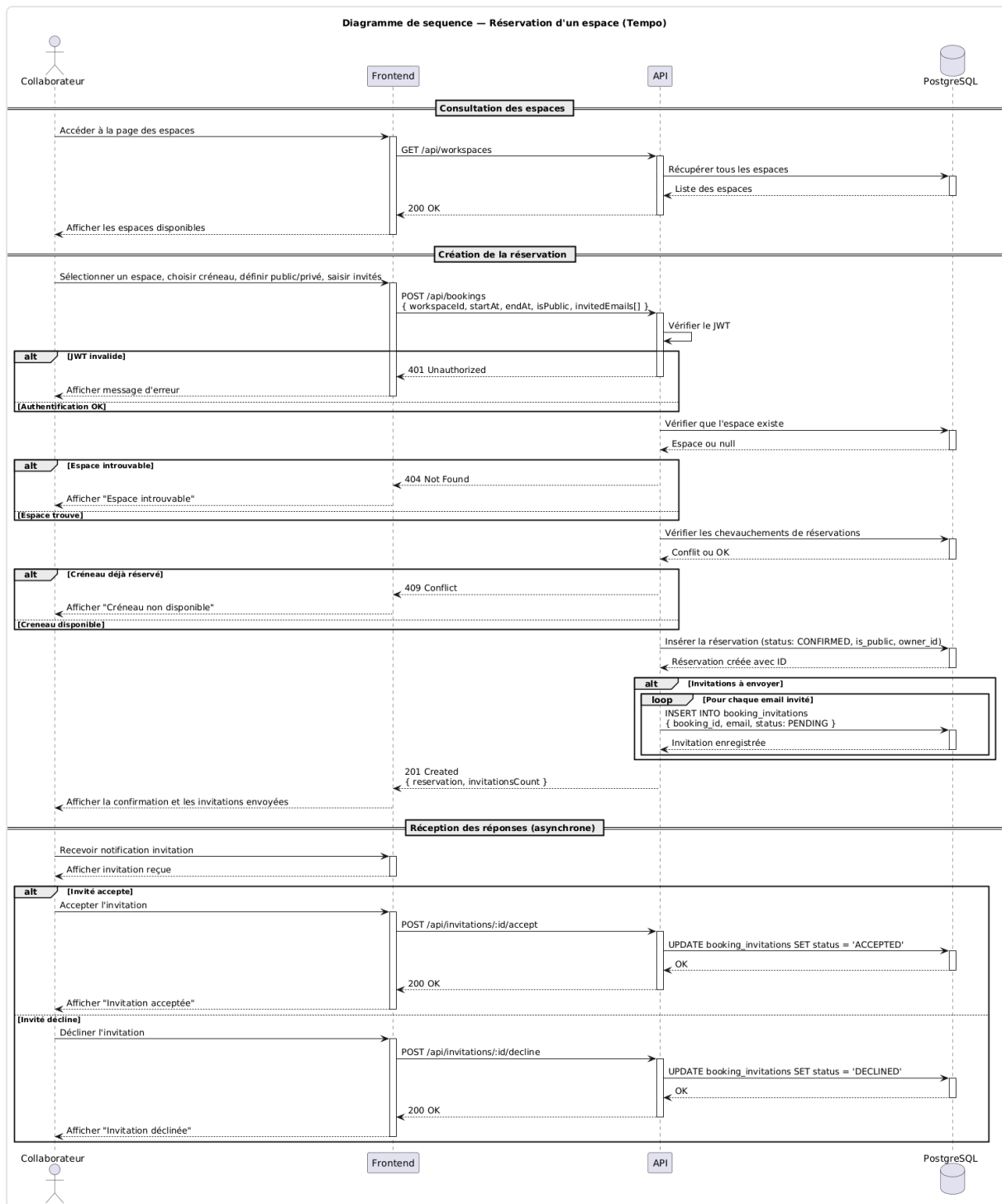
3.2.1. Réservation d'un espace

Le collaborateur filtre les espaces disponibles, choisit un créneau et crée une réservation publique ou privée. Il peut ensuite inviter des collaborateurs. Le système vérifie les chevauchements et le quota.

Diagramme d'activite - Reservation d'un espace (Tempo)



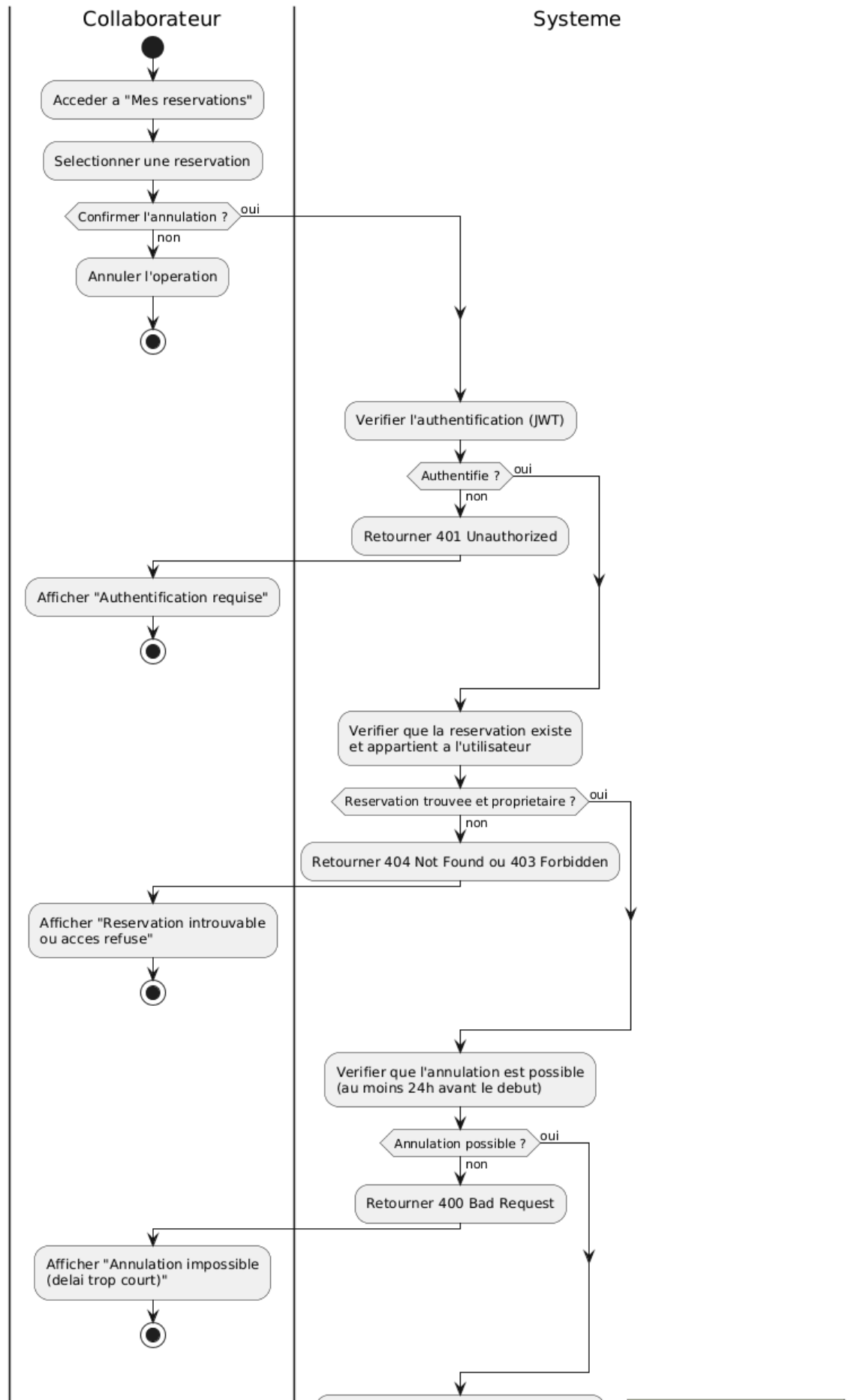


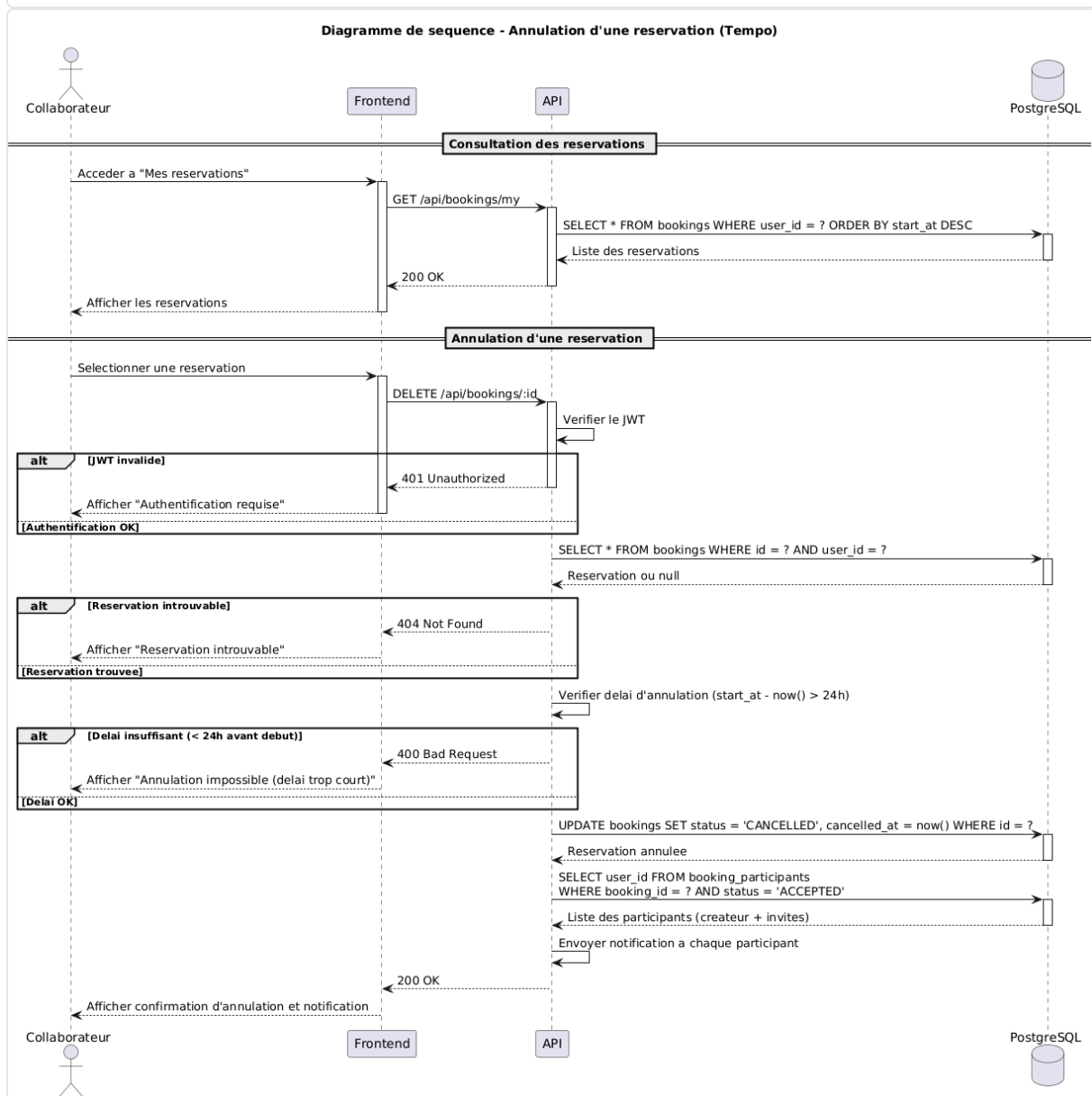
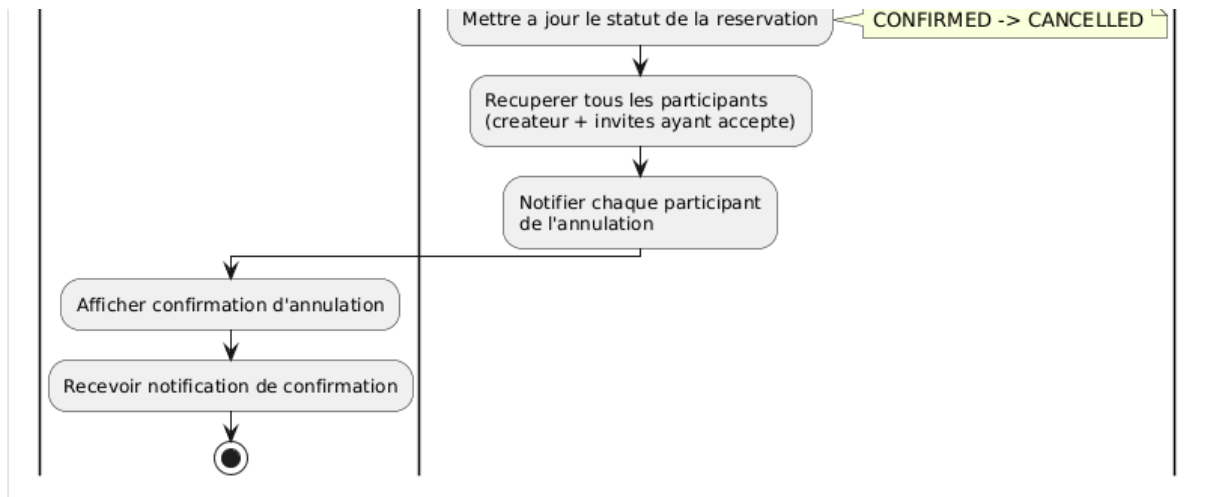


3.2.2. Annulation d'une réservation

Le collaborateur annule sa réservation (min. 24h avant le début). Le système passe le statut à **CANCELLED** et notifie tous les participants (créateur + invités acceptés).

Diagramme d'activite - Annulation d'une reservation (Tempo)

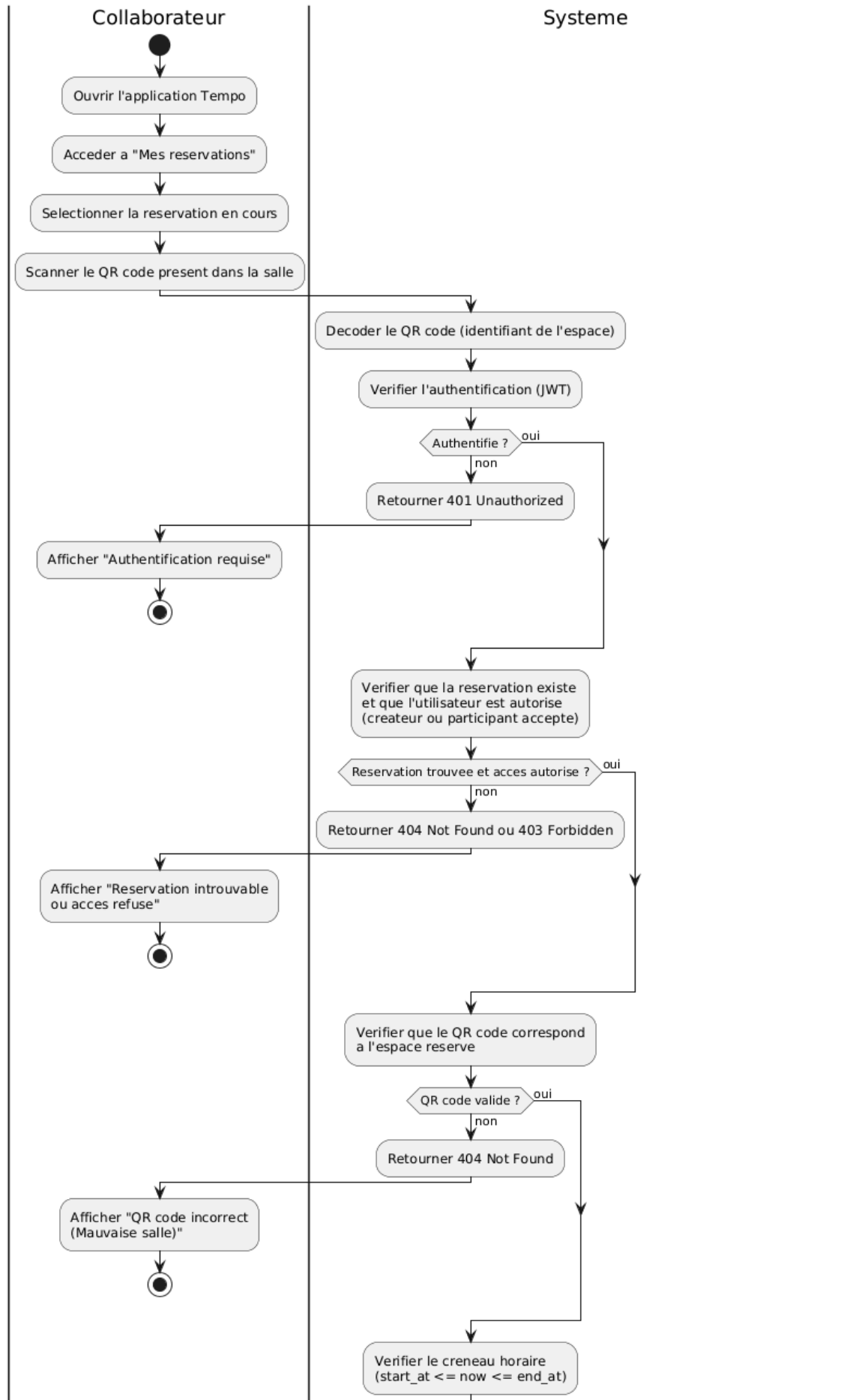


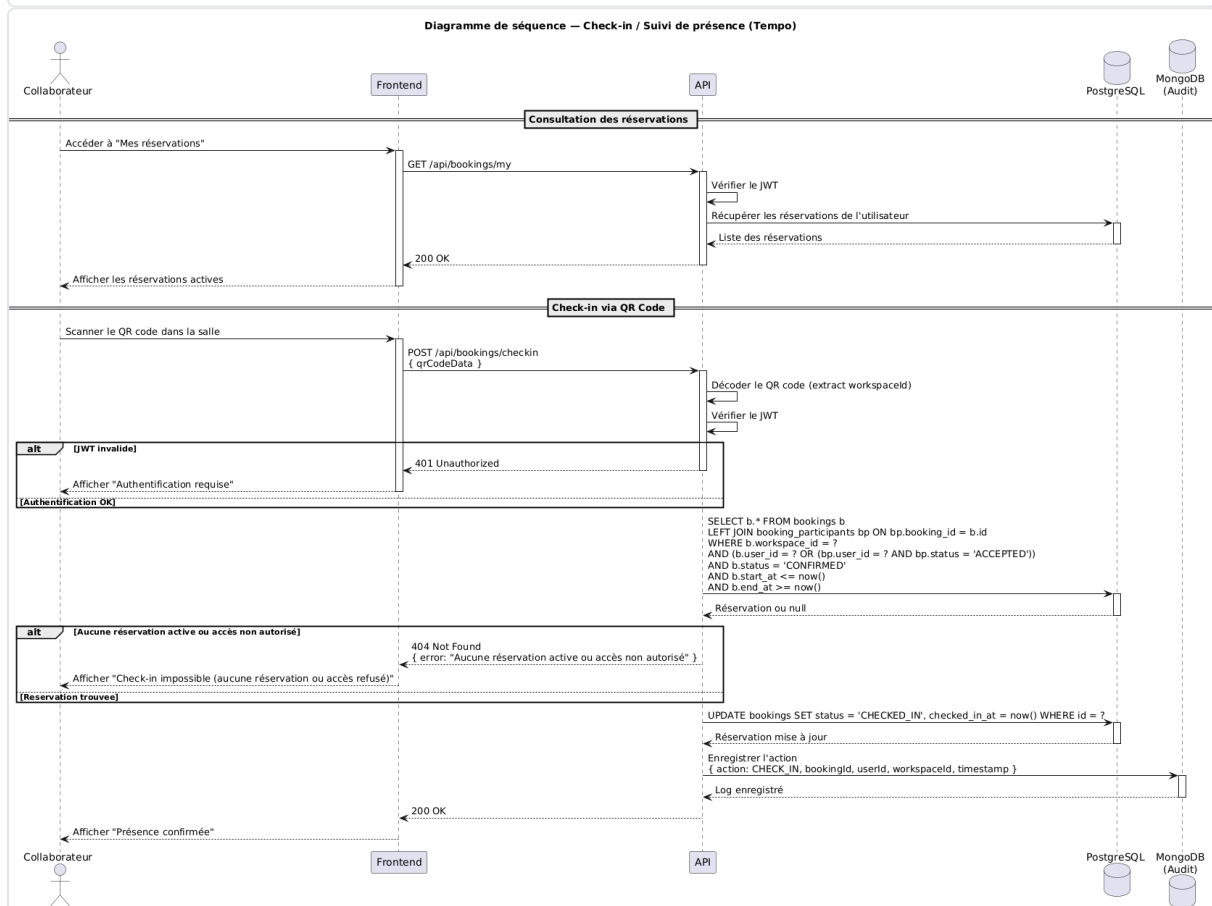
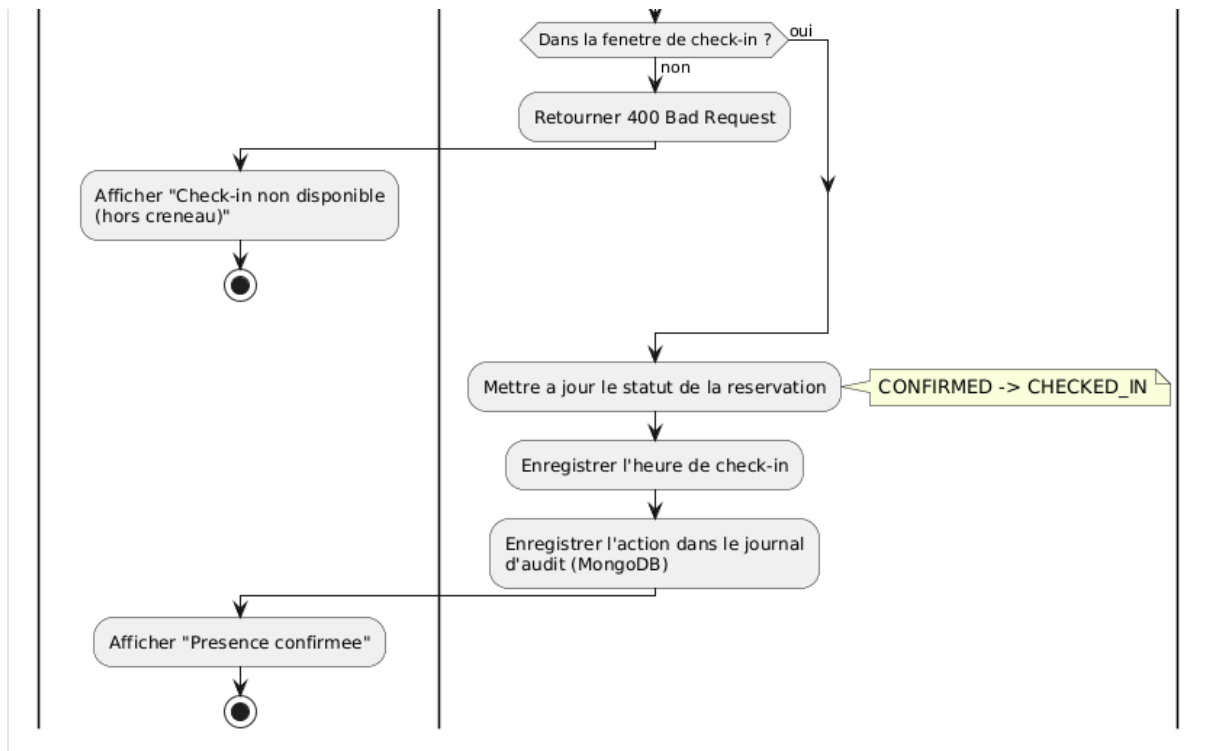


3.2.3. Check-in / Suivi de présence

Le collaborateur (créateur ou participant accepté) scanne le QR code dans la salle pendant son créneau. Le système vérifie la correspondance espace/réservation et passe le statut à **CHECKED_IN**. L'action est tracée en audit (MongoDB).

Diagramme d'activite - Check-in / Suivi de presence (Tempo)

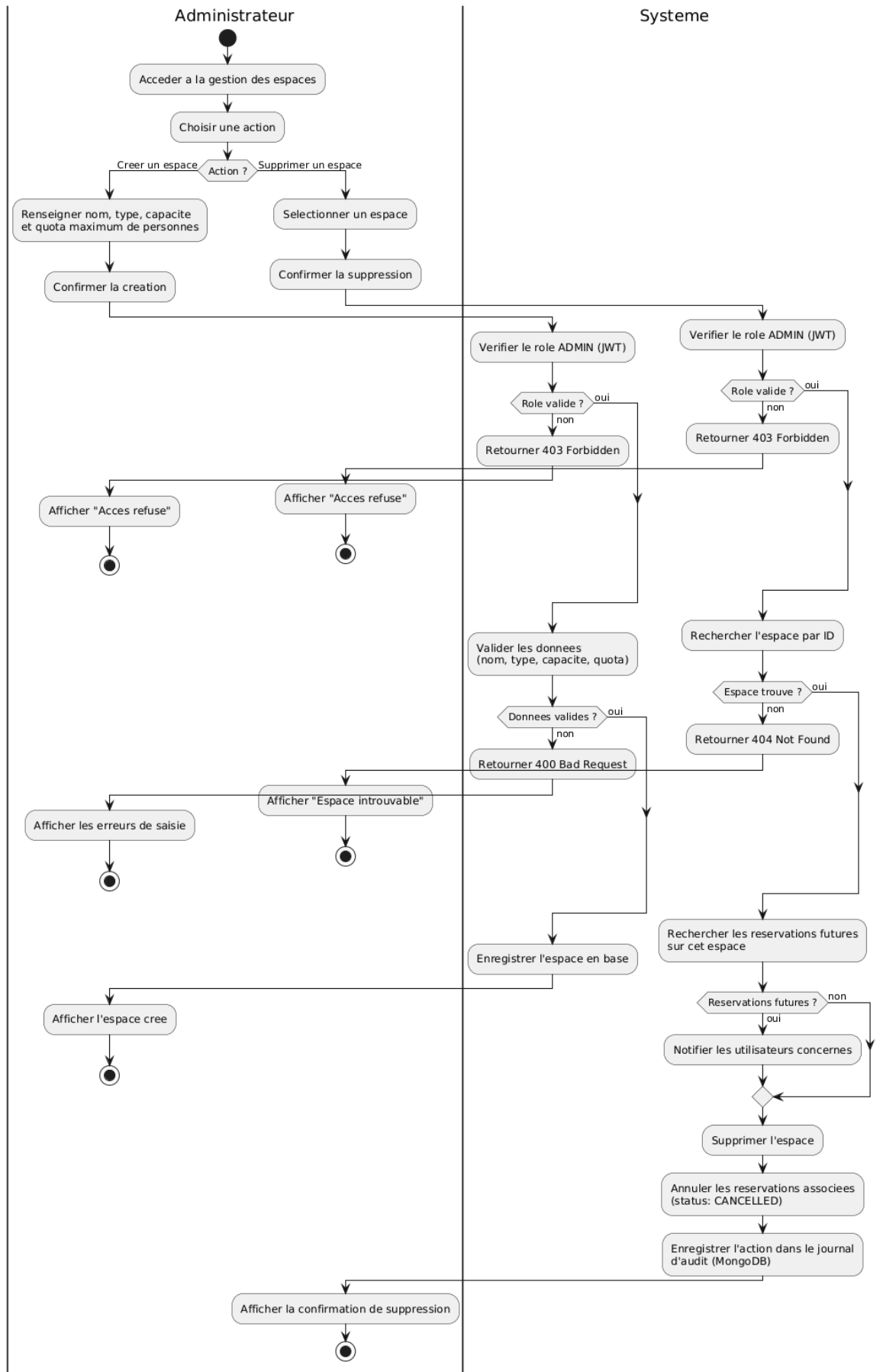


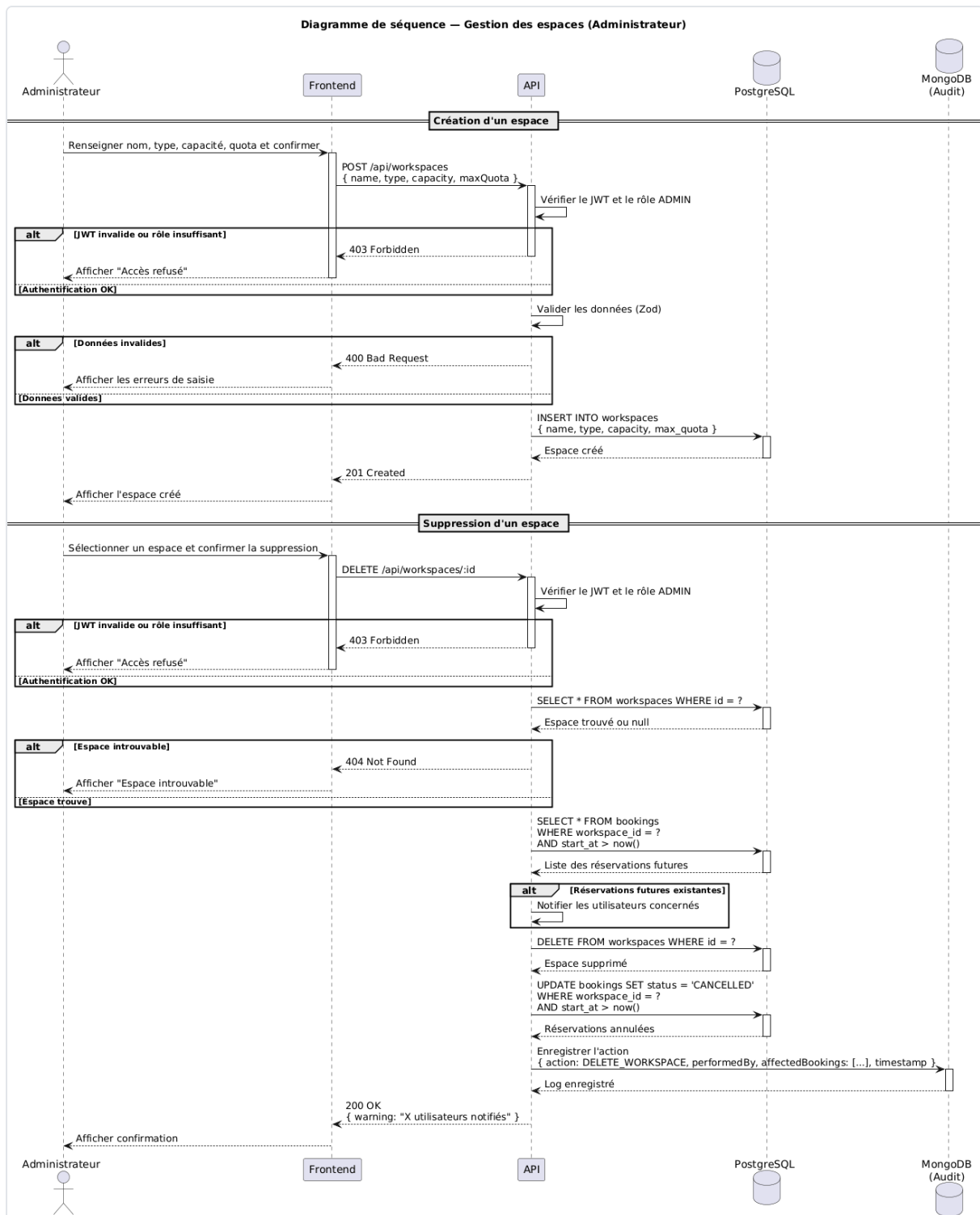


3.2.4. Gestion des espaces (Admin)

L'administrateur crée, modifie ou supprime des espaces. La suppression annule automatiquement les réservations futures et notifie les utilisateurs concernés.

Diagramme d'activité - Gestion des espaces (Administrateur)

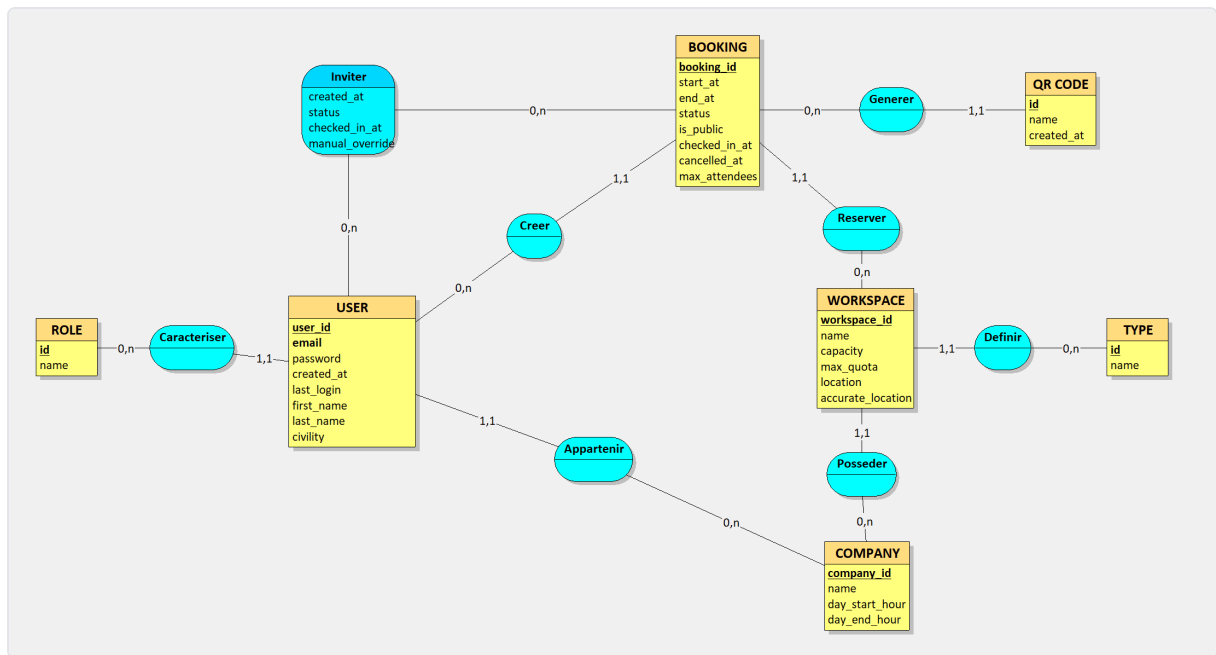




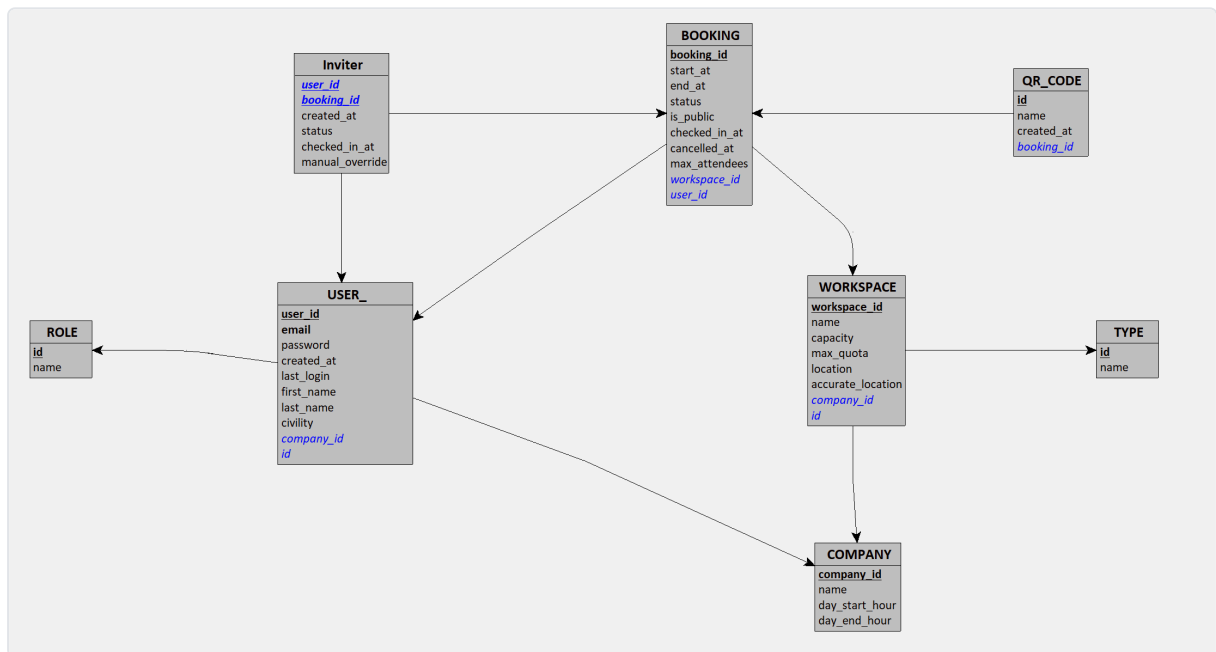
4. Conception modèle de données

4.1. MCD (MERISE)

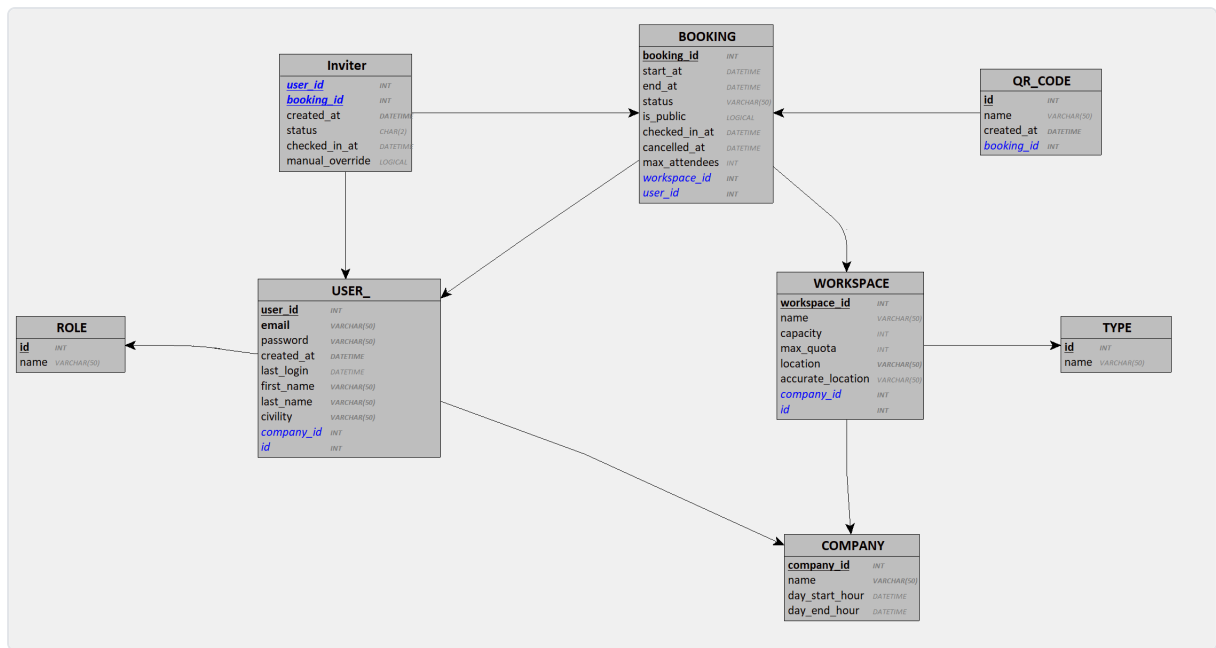
Réalisé avec [Looping](#) (fichier source : `diagrams/merise/Looping1.luo`).



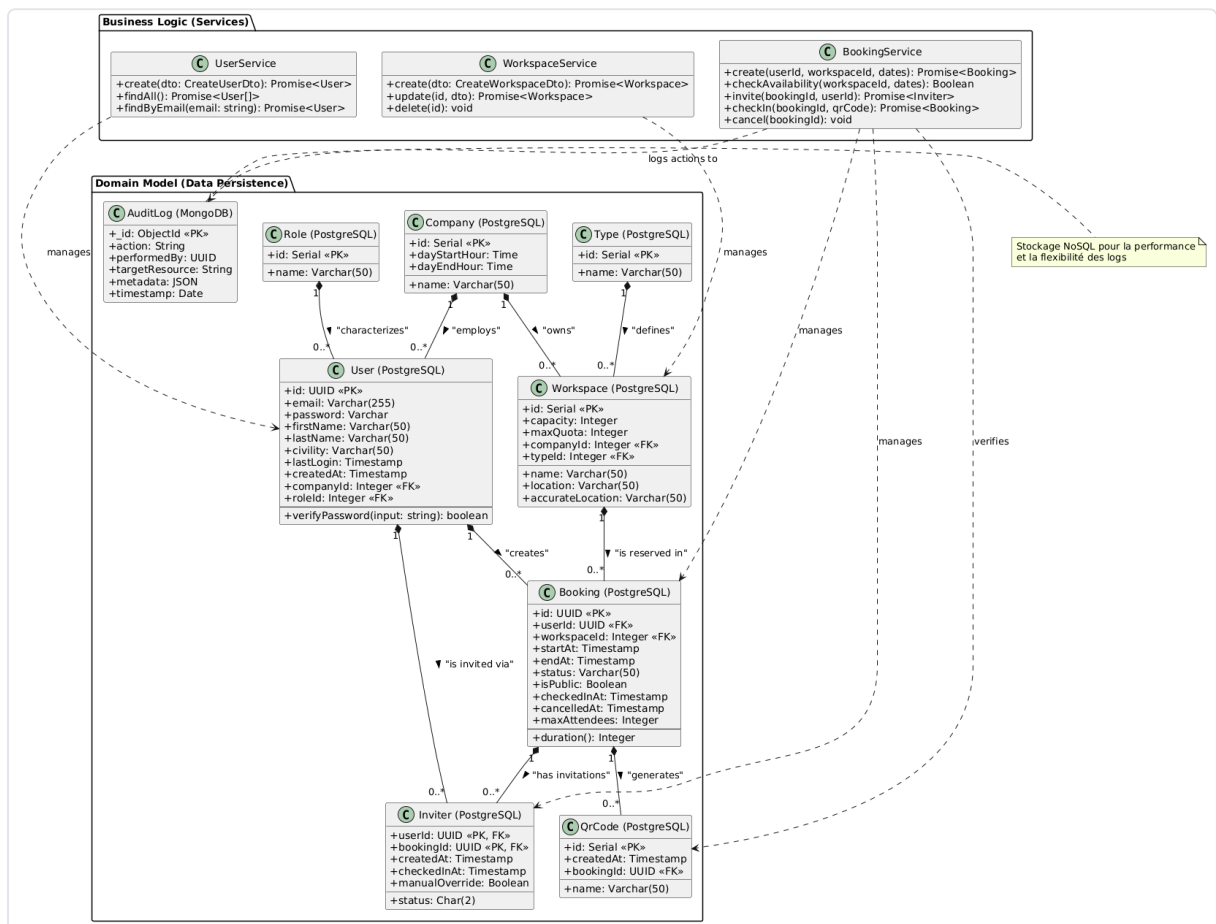
4.2. MLD (MERISE)



4.3. MPD (MERISE)



4.4. Diagramme de classes (UML)



4.5. Script de création de la base de données

Généré par **Drizzle Kit** (`bunx drizzle-kit generate`) à partir du schéma TypeScript :


```

-- Enumérations
CREATE TYPE "public"."role" AS ENUM('ADMIN', 'USER');
CREATE TYPE "public"."workspace_type" AS ENUM('DESK', 'MEETING_ROOM');

-- Table users
CREATE TABLE "users" (
  "id" uuid PRIMARY KEY DEFAULT gen_random_uuid() NOT NULL,
  "email" text NOT NULL,
  "password" text NOT NULL,
  "role" "role" DEFAULT 'USER',
  "created_at" timestamp DEFAULT now(),
  CONSTRAINT "users_email_unique" UNIQUE("email")
);

-- Table workspaces
CREATE TABLE "workspaces" (
  "id" serial PRIMARY KEY NOT NULL,
  "name" text NOT NULL,
  "type" "workspace_type" NOT NULL,
  "capacity" integer DEFAULT 1 NOT NULL,
  "created_at" timestamp DEFAULT now()
);

-- Table bookings
CREATE TABLE "bookings" (
  "id" uuid PRIMARY KEY DEFAULT gen_random_uuid() NOT NULL,
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE cascade,
  "workspace_id" integer NOT NULL REFERENCES "workspaces"("id") ON DELETE cascade,
  "start_at" timestamp NOT NULL,
  "end_at" timestamp NOT NULL,
  "created_at" timestamp DEFAULT now()
);

```

Argumentation : Drizzle ORM génère des migrations TypeScript-first versionnées dans `apps/backend/drizzle/`. Les UUIDs pour `users` et `bookings` empêchent l'énumération d'IDs. Les clés étrangères utilisent `ON DELETE CASCADE` pour maintenir l'intégrité référentielle.

4.6. Script de création (complément MPD)

Pour aligner le schéma physique avec le MPD complet (sections 4.1-4.3), les migrations suivantes seraient nécessaires :

```

-- Tables de référence
CREATE TABLE "roles" (
  "id" serial PRIMARY KEY NOT NULL,

```

```

        "name" varchar(50) NOT NULL UNIQUE
    );

CREATE TABLE "companies" (
    "id" serial PRIMARY KEY NOT NULL,
    "name" varchar(50) NOT NULL,
    "day_start_hour" time NOT NULL,
    "day_end_hour" time NOT NULL
);

CREATE TABLE "types" (
    "id" serial PRIMARY KEY NOT NULL,
    "name" varchar(50) NOT NULL UNIQUE
);

-- Enrichissement de users (profil + rattachements)
ALTER TABLE "users" ADD COLUMN "first_name" varchar(50);
ALTER TABLE "users" ADD COLUMN "last_name" varchar(50);
ALTER TABLE "users" ADD COLUMN "civility" varchar(50);
ALTER TABLE "users" ADD COLUMN "last_login" timestamp;
ALTER TABLE "users" ADD COLUMN "company_id" integer REFERENCES "companies"("id");
ALTER TABLE "users" ADD COLUMN "role_id" integer REFERENCES "roles"("id");

-- Enrichissement de workspaces (localisation + rattachements)
ALTER TABLE "workspaces" ADD COLUMN "max_quota" integer;
ALTER TABLE "workspaces" ADD COLUMN "location" varchar(50);
ALTER TABLE "workspaces" ADD COLUMN "accurate_location" varchar(50);
ALTER TABLE "workspaces" ADD COLUMN "company_id" integer REFERENCES "companies"("id");
ALTER TABLE "workspaces" ADD COLUMN "type_id" integer REFERENCES "types"("id");

-- Enrichissement de bookings (cycle de vie)
ALTER TABLE "bookings" ADD COLUMN "status" varchar(50) DEFAULT 'CONFIRMED';
ALTER TABLE "bookings" ADD COLUMN "is_public" boolean DEFAULT false;
ALTER TABLE "bookings" ADD COLUMN "checked_in_at" timestamp;
ALTER TABLE "bookings" ADD COLUMN "cancelled_at" timestamp;
ALTER TABLE "bookings" ADD COLUMN "max_attendees" integer;
ALTER TABLE "bookings" ADD CONSTRAINT "valid_dates" CHECK ("end_at" > "start_at");

-- QR codes générés par réservation
CREATE TABLE "qr_codes" (
    "id" serial PRIMARY KEY NOT NULL,
    "name" varchar(50) NOT NULL,
    "created_at" timestamp DEFAULT now(),
    "booking_id" uuid NOT NULL REFERENCES "bookings"("id") ON DELETE cascade
);

```

4.7. Script de modification (exemple de migration)

Exemple concret : ajout du système d'invitations (table issue de l'association porteuse **Inviter**) dans une migration ultérieure.

Fichier : `apps/backend/drizzle/0001_add_inviter.sql`

```
-- Table de liaison n,n entre users et bookings (association porteuse Inviter)
CREATE TABLE "inviter" (
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE cascade,
  "booking_id" uuid NOT NULL REFERENCES "bookings"("id") ON DELETE cascade,
  "created_at" timestamp NOT NULL DEFAULT now(),
  "status" char(2) NOT NULL,
  "checked_in_at" timestamp,
  "manual_override" boolean NOT NULL DEFAULT false,
  CONSTRAINT "inviter_pk" PRIMARY KEY ("user_id", "booking_id")
);

-- Index pour optimiser les recherches
CREATE INDEX "idx_inviter_booking_id" ON "inviter" ("booking_id");
CREATE INDEX "idx_inviter_status" ON "inviter" ("status");
```

Commandes :

```
# Générer la migration depuis schema.ts
bunx drizzle-kit generate

# Appliquer les migrations en base
bunx drizzle-kit migrate
```

5. Environnement technique

Domaine	Technologie	Justification
Runtime	Bun	Ultra-rapide, remplace Node + npm, test runner intégré
Backend	Hono	Ultra-léger, typage RPC natif (AppType)
Frontend	Svelte 5	Runes (\$state , \$derived), sans Virtual DOM
Style	Tailwind CSS 4 + shadcn-svelte	Utility-first, composants accessibles
Base SQL	PostgreSQL 18 + Drizzle ORM	Intégrité relationnelle, migrations type-safe

Domaine	Technologie	Justification
Base NoSQL	MongoDB 8	Logs d'audit : volume variable, schéma flexible
Auth	JWT (hono/jwt)	Stateless, sécurisation de toutes les routes
Qualité	Oxlint + Oxfmt + EditorConfig	Lint et formatter Rust ultra-rapide, tabs 4, guillemets simples
Tests	Bun Test (backend) + Vitest (frontend)	Intégrés aux runtimes respectifs
CI/CD	GitHub Actions	Lint + test + build à chaque push
Conteneurs	Docker + Docker Compose	Multi-stage builds, reproductibilité locale et prod
Gestion projet	Kanban (Trello)	Suivi simple de l'avancement

Ports : Backend :3000 · Frontend :5173 · PostgreSQL :5432 · MongoDB :27017